LEETCODE

We identified Scheduling, Synchronization and Memory Management exercises for Additional exercises. Comments will be written by instructor in the box "Instructor comments" - comments for solving the problems.

Ex 4 :

Single Threaded CPU

1)You are given n tasks labeled from 0 to n - 1 represented by a 2D integer array tasks, where tasks[i] = [enqueueTimeᵢ, processingTimeᵢ] means that the ith task will be available to process at enqueueTimeᵢ and will take processingTimeᵢ to finish processing.

You have a single-threaded CPU that can process **at most one** task at a time and will act in the following way:

- If the CPU is idle and there are no available tasks to process, the CPU remains idle.
- If the CPU is idle and there are available tasks, the CPU will choose the one with the **shortest processing time**. If multiple tasks have the same shortest processing time, it will choose the task with the smallest index.
- Once a task is started, the CPU will **process the entire task** without stopping.
- The CPU can finish a task then start a new one instantly.

Return the order in which the CPU will process the tasks.

Program:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int start, duration, index;
} Task;

int compare(const void *a, const void *b) {
    const Task *x = a, *y = b;

    if (x->start != y->start)
        return (x->start > y->start) - (x->start < y->start);

    return (x->index > y->index) - (x->index < y->index);
}
```

```
int less(Task a, Task b) {
    if (a.duration != b.duration)
        return a.duration < b.duration;

    return a.index < b.index;
}

void push(Task heap[], int *size, Task t) {
    int i = (*size)++;

    while (i > 0) {
        int p = (i - 1) / 2;

        if (!less(t, heap[p]))
            break;

        heap[i] = heap[p];
        i = p;
    }

    heap[i] = t;
}

Task pop(Task heap[], int *size) {
    Task result = heap[0];
    Task last = heap[--(*size)];

    if (*size == 0)
        return result;

    int i = 0;

    while (2 * i + 1 < *size) {
        int child = 2 * i + 1;

        if (child + 1 < *size &&
```

```
            less(heap[child + 1], heap[child]))
            child++;

        if (!less(heap[child], last))
            break;

        heap[i] = heap[child];
        i = child;
    }

    heap[i] = last;
    return result;
}

int* getOrder(int** tasks, int tasksSize,
              int* tasksColSize, int* returnSize) {

    int n = tasksSize;

    Task *arr = malloc(n * sizeof(Task));
    Task *heap = malloc(n * sizeof(Task));
    int *answer = malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
        arr[i] = (Task){
            tasks[i][0],
            tasks[i][1],
            i
        };
    }

    qsort(arr, n, sizeof(Task), compare);

    long long time = 0;
    int i = 0, size = 0, count = 0;
```

```
    while (count < n) {

        if (size == 0 && time < arr[i].start)
            time = arr[i].start;

        while (i < n && arr[i].start <= time)
            push(heap, &size, arr[i++]);

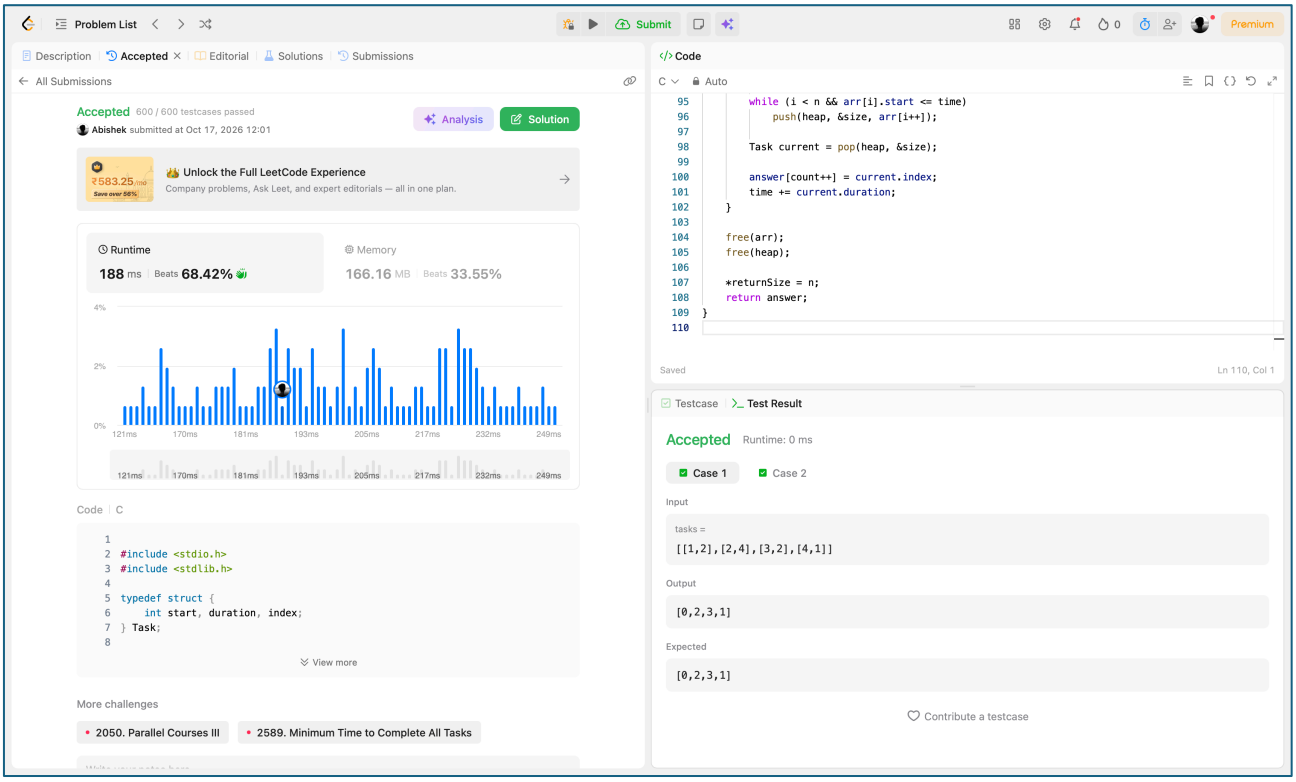
        Task current = pop(heap, &size);

        answer[count++] = current.index;
        time += current.duration;
    }

    free(arr);
    free(heap);

    *returnSize = n;
    return answer;
}
```

Output:



2) Process Task using Servers

You are given two **0-indexed** integer arrays servers and tasks of lengths n and m respectively. servers[i] is the **weight** of the ith server, and tasks[j] is the **time needed** to process the jth task **in seconds**.

Tasks are assigned to the servers using a **task queue**. Initially, all servers are free, and the queue is **empty**.

At second j, the jth task is **inserted** into the queue (starting with the 0th task being inserted at second 0). As long as there are free servers and the queue is not empty, the task in the front of the queue will be assigned to a free server with the **smallest weight**, and in case of a tie, it is assigned to a free server with the **smallest index**.

If there are no free servers and the queue is not empty, we wait until a server becomes free and immediately assign the next task. If multiple servers become free at the same time, then multiple tasks from the queue will be assigned **in order of insertion** following the weight and index priorities above.

A server that is assigned task j at second t will be free again at second t + tasks[j].

Build an array ans of length m, where ans[j] is the **index** of the server the jth task will be assigned to.

Return *the array* ans.

Program:

```
#include <pthread.h>
#include <stdlib.h>

typedef struct {
    pthread_mutex_t forks[5];
} DiningPhilosophers;

DiningPhilosophers* diningPhilosophersCreate() {
    DiningPhilosophers* obj = malloc(sizeof(DiningPhilosophers));

    for (int i = 0; i < 5; i++) {
        pthread_mutex_init(&obj->forks[i], NULL);
    }

    return obj;
}
```

```
void wantsToEat(
    DiningPhilosophers* obj,
    int philosopher,
    void (*pickLeftFork)(),
    void (*pickRightFork)(),
    void (*eat)(),
    void (*putLeftFork)(),
    void (*putRightFork)()
) {
    int left = philosopher;
    int right = (philosopher + 1) % 5;

    int first = left < right ? left : right;
    int second = left < right ? right : left;

    pthread_mutex_lock(&obj->forks[first]);
    pthread_mutex_lock(&obj->forks[second]);

    pickLeftFork();
    pickRightFork();

    eat();

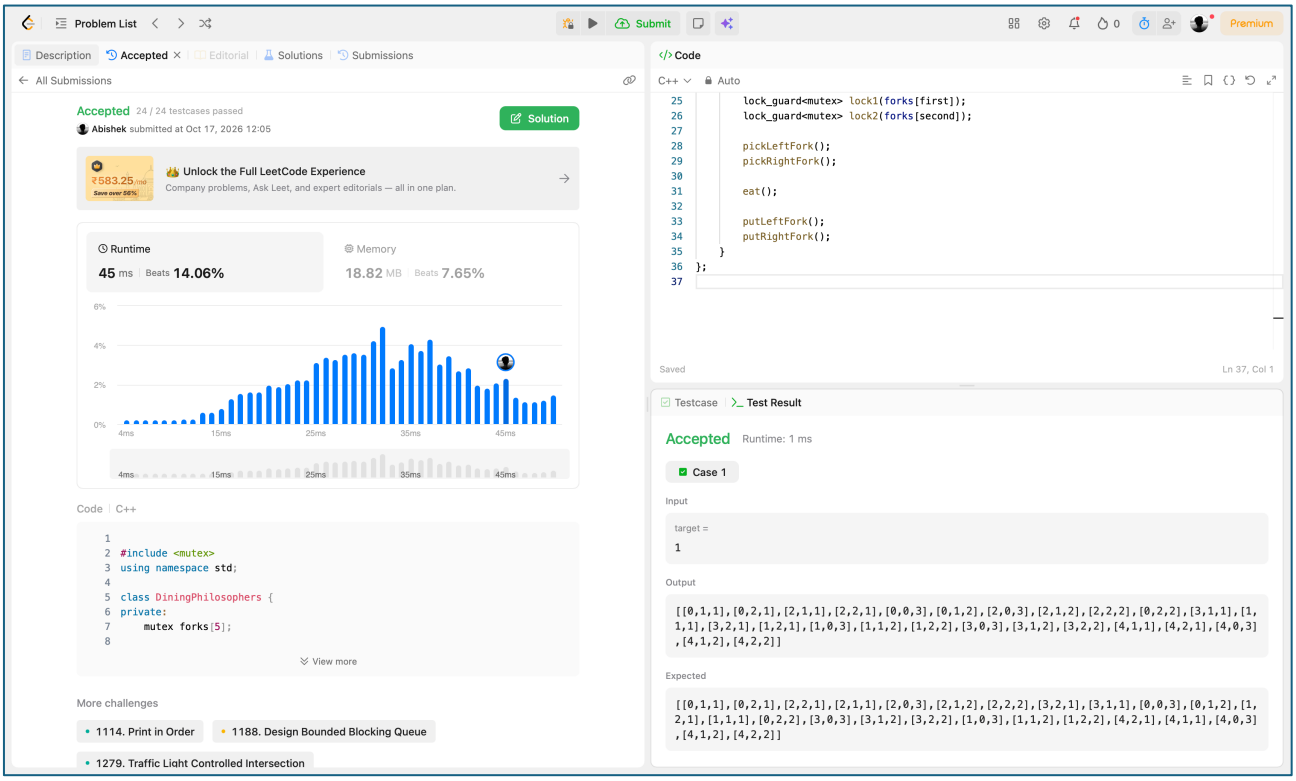
    putLeftFork();
    putRightFork();

    pthread_mutex_unlock(&obj->forks[second]);
    pthread_mutex_unlock(&obj->forks[first]);
}

void diningPhilosophersFree(DiningPhilosophers* obj) {
    for (int i = 0; i < 5; i++) {
        pthread_mutex_destroy(&obj->forks[i]);
    }

    free(obj);}
```

Output:



3)Task Scheduler

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of **at least** n intervals between two tasks with the same label.

Return the **minimum** number of CPU intervals required to complete all tasks.

Program:

```
int leastInterval(char* tasks, int tasksSize, int n) {
    int freq[26] = {0};

    for (int i = 0; i < tasksSize; i++) {
        freq[tasks[i] - 'A']++;
    }

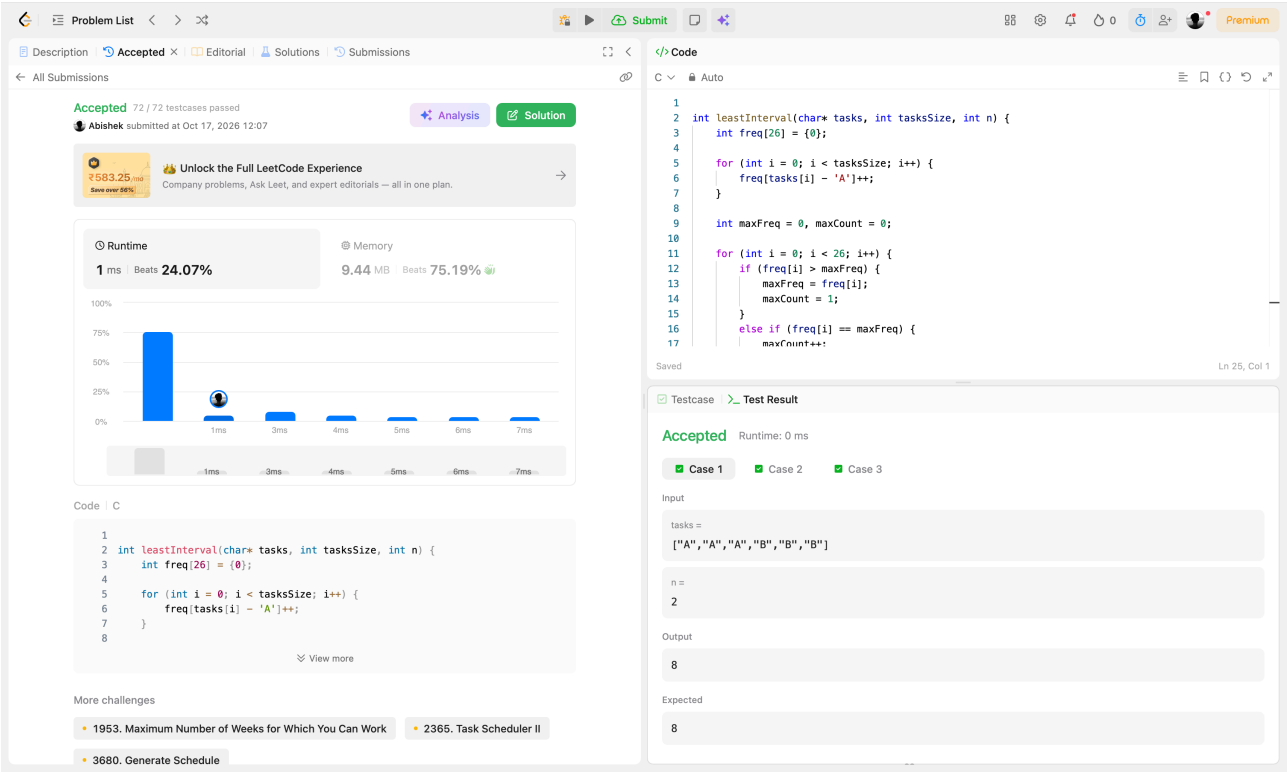
    int maxFreq = 0, maxCount = 0;

    for (int i = 0; i < 26; i++) {
        if (freq[i] > maxFreq) {
            maxFreq = freq[i];
            maxCount = 1;
        }
        else if (freq[i] == maxFreq) {
            maxCount++;
        }
    }

    int minTime = (maxFreq - 1) * (n + 1) + maxCount;

    return tasksSize > minTime ? tasksSize : minTime;}
```

Output:



Ex 5:

4) The Dining Philosophers

Five silent philosophers sit at a round table with bowls of spaghetti. Forks are placed between each pair of adjacent philosophers.

Each philosopher must alternately think and eat. However, a philosopher can only eat spaghetti when they have both left and right forks. Each fork can be held by only one philosopher and so a philosopher can use the fork only if it is not being used by another philosopher. After an individual philosopher finishes eating, they need to put down both forks so that the forks become available to others. A philosopher can take the fork on their right or the one on their left as they become available, but cannot start eating before getting both forks.

Eating is not limited by the remaining amounts of spaghetti or stomach space; an infinite supply and an infinite demand are assumed.

Design a discipline of behaviour (a concurrent algorithm) such that no philosopher will starve; *i.e.*, each can forever continue to alternate between eating and thinking, assuming that no philosopher can know when others may want to eat or think.

Program:

```
#include <stdlib.h>

typedef struct {

    int weight;

    int index;
```

```
    long long time;
} Server;

int availableLess(Server a, Server b) {
    if (a.weight != b.weight)
        return a.weight < b.weight;
    return a.index < b.index;
}

int busyLess(Server a, Server b) {
    if (a.time != b.time)
        return a.time < b.time;
    if (a.weight != b.weight)
        return a.weight < b.weight;
    return a.index < b.index;
}

void push(Server heap[], int *size, Server s,
          int (*less)(Server, Server)) {
    int i = (*size)++;

    while (i > 0) {
        int p = (i - 1) / 2;

        if (!less(s, heap[p]))
            break;

        heap[i] = heap[p];
        i = p;
    }

    heap[i] = s;
}

Server pop(Server heap[], int *size,
           int (*less)(Server, Server)) {
```

```
    Server result = heap[0];
    Server last = heap[--(*size)];

    if (*size == 0)
        return result;

    int i = 0;

    while (2 * i + 1 < *size) {
        int child = 2 * i + 1;

        if (child + 1 < *size &&
            less(heap[child + 1], heap[child]))
            child++;

        if (!less(heap[child], last))
            break;

        heap[i] = heap[child];
        i = child;
    }

    heap[i] = last;
    return result;
}

int* assignTasks(int* servers, int serversSize,
                 int* tasks, int tasksSize,
                 int* returnSize) {

    int n = serversSize;
    int m = tasksSize;

    Server *available = malloc(n * sizeof(Server));
    Server *busy = malloc(n * sizeof(Server));
    int *ans = malloc(m * sizeof(int));
```

```
int freeSize = 0, busySize = 0;
long long time = 0;

for (int i = 0; i < n; i++) {
    Server s = {servers[i], i, 0};
    push(available, &freeSize, s, availableLess);
}

for (int i = 0; i < m; i++) {
    if (time < i)
        time = i;

    if (freeSize == 0 && time < busy[0].time)
        time = busy[0].time;

    while (busySize > 0 && busy[0].time <= time) {
        Server s = pop(busy, &busySize, busyLess);
        push(available, &freeSize, s, availableLess);
    }

    Server s = pop(available, &freeSize, availableLess);

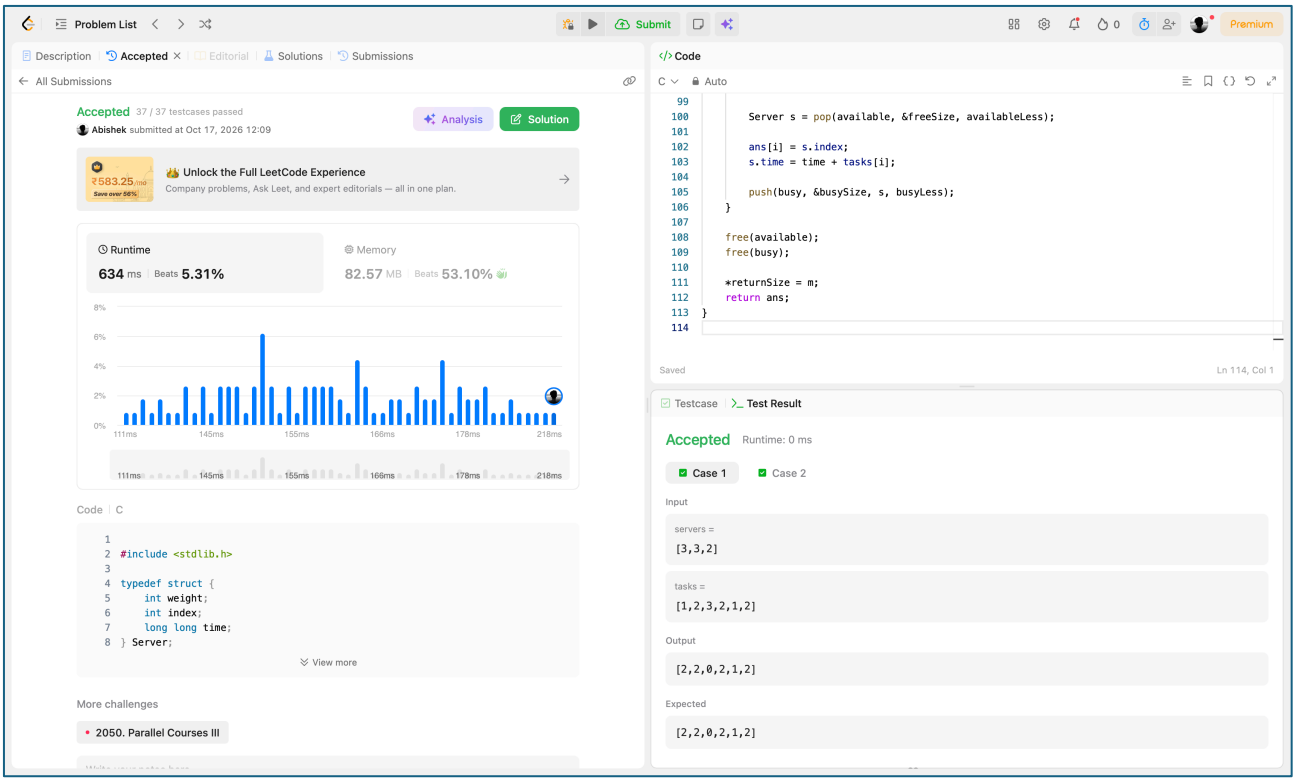
    ans[i] = s.index;
    s.time = time + tasks[i];

    push(busy, &busySize, s, busyLess);
}

free(available);
free(busy);

*returnSize = m;
return ans;}
```

Output:



Ex 6:

5) Building H2O

There are two kinds of threads: oxygen and hydrogen. Your goal is to group these threads to form water molecules.

There is a barrier where each thread has to wait until a complete molecule can be formed. Hydrogen and oxygen threads will be given releaseHydrogen and releaseOxygen methods respectively, which will allow them to pass the barrier. These threads should pass the barrier in groups of three, and they must immediately bond with each other to form a water molecule. You must guarantee that all the threads from one molecule bond before any other threads from the next molecule do.

In other words:

- If an oxygen thread arrives at the barrier when no hydrogen threads are present, it must wait for two hydrogen threads.
- If a hydrogen thread arrives at the barrier when no other threads are present, it must wait for an oxygen thread and another hydrogen thread.

We do not have to worry about matching the threads up explicitly; the threads do not necessarily know which other threads they are paired up with. The key is that threads pass the barriers in complete sets; thus, if we examine the sequence of threads that bind and divide them into groups of three, each group should contain one oxygen and two hydrogen threads.

Write synchronization code for oxygen and hydrogen molecules that enforces these constraints.

Program:

```
#include <semaphore.h>
#include <stdlib.h>

typedef struct {
    sem_t s1;
    sem_t s2;
} Foo;

Foo* fooCreate() {
    Foo* obj = malloc(sizeof(Foo));

    sem_init(&obj->s1, 0, 0);
    sem_init(&obj->s2, 0, 0);

    return obj;
}

void first(Foo* obj, void (*printFirst)()) {
    printFirst();
    sem_post(&obj->s1);
}

void second(Foo* obj, void (*printSecond)()) {
    sem_wait(&obj->s1);
    printSecond();
    sem_post(&obj->s2);
}

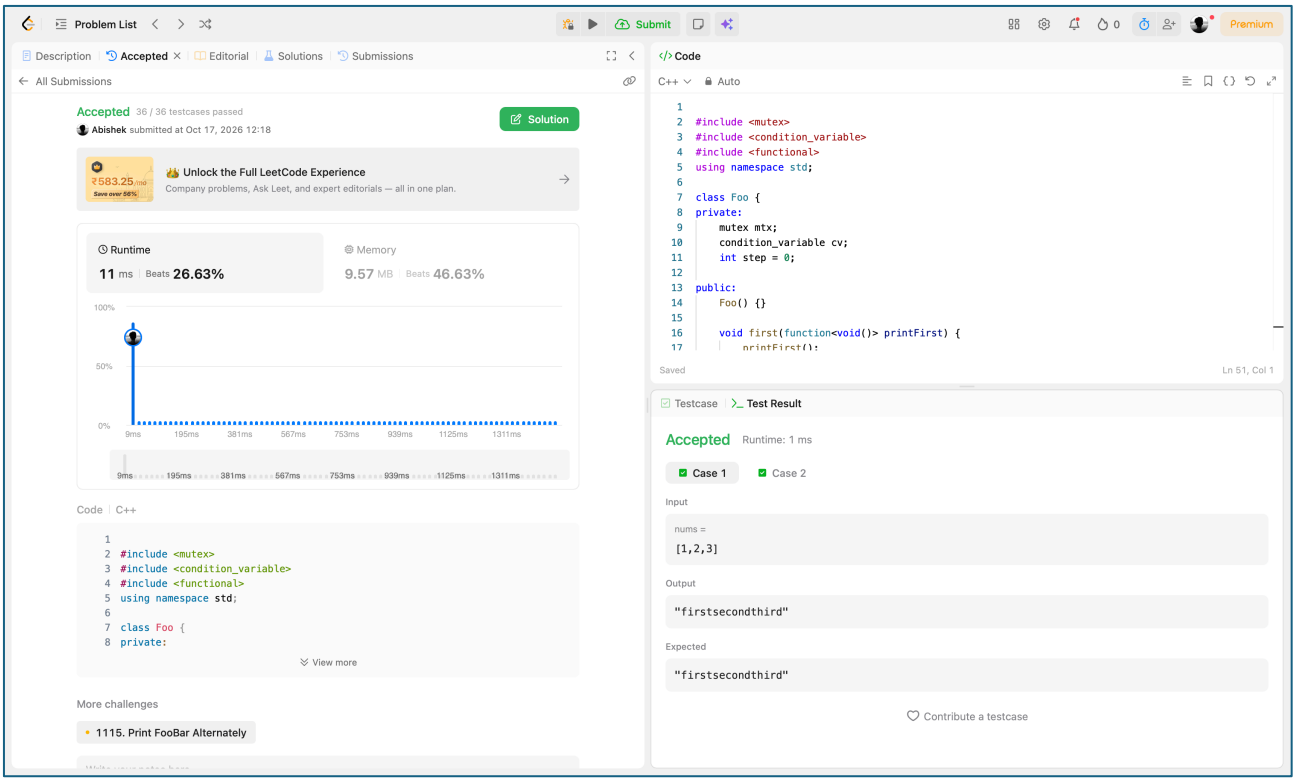
void third(Foo* obj, void (*printThird)()) {
    sem_wait(&obj->s2);
    printThird();
}

void fooFree(Foo* obj) {
    sem_destroy(&obj->s1);
    sem_destroy(&obj->s2);
    free(obj);
}
6) Print in order

Suppose we have a class:

public class Foo {
  public void first() { print("first"); }
  public void second() { print("second"); }
  public void third() { print("third"); }
}
```

Output:



The same instance of Foo will be passed to three different threads. Thread A will call first(), thread B will call second(), and thread C will call third(). Design a mechanism and modify the program to ensure that second() is executed after first(), and third() is executed after second().

Note:

We do not know how the threads will be scheduled in the operating system, even though the numbers in the input seem to imply the ordering. The input format you see is mainly to ensure our tests' comprehensiveness.

Program:

```
#include <stdlib.h>

typedef struct Node {
    int key, value;
    struct Node *prev, *next;
} Node;

typedef struct {
    int capacity, size;
    Node *head, *tail;
    Node *map[10001];
} LRUCache;

void removeNode(Node *node) {
    node->prev->next = node->next;
    node->next->prev = node->prev;
}

void addFront(LRUCache *obj, Node *node) {
```

```
    node->next = obj->head->next;
    node->prev = obj->head;

    obj->head->next->prev = node;
    obj->head->next = node;
}

void moveFront(LRUCache *obj, Node *node) {
    removeNode(node);
    addFront(obj, node);
}

LRUCache* lRUCacheCreate(int capacity) {
    LRUCache *obj = calloc(1, sizeof(LRUCache));

    obj->capacity = capacity;

    obj->head = calloc(1, sizeof(Node));
    obj->tail = calloc(1, sizeof(Node));

    obj->head->next = obj->tail;
    obj->tail->prev = obj->head;

    return obj;
}

int lRUCacheGet(LRUCache* obj, int key) {
    Node *node = obj->map[key];

    if (node == NULL)
        return -1;

    moveFront(obj, node);
    return node->value;
}

void lRUCachePut(LRUCache* obj, int key, int value) {
    Node *node = obj->map[key];

    if (node != NULL) {
        node->value = value;
        moveFront(obj, node);
        return;
    }

    if (obj->size == obj->capacity) {
        Node *lru = obj->tail->prev;

        removeNode(lru);
        obj->map[lru->key] = NULL;

        free(lru);
        obj->size--;
    }

    node = malloc(sizeof(Node));
    node->key = key;
    node->value = value;
```

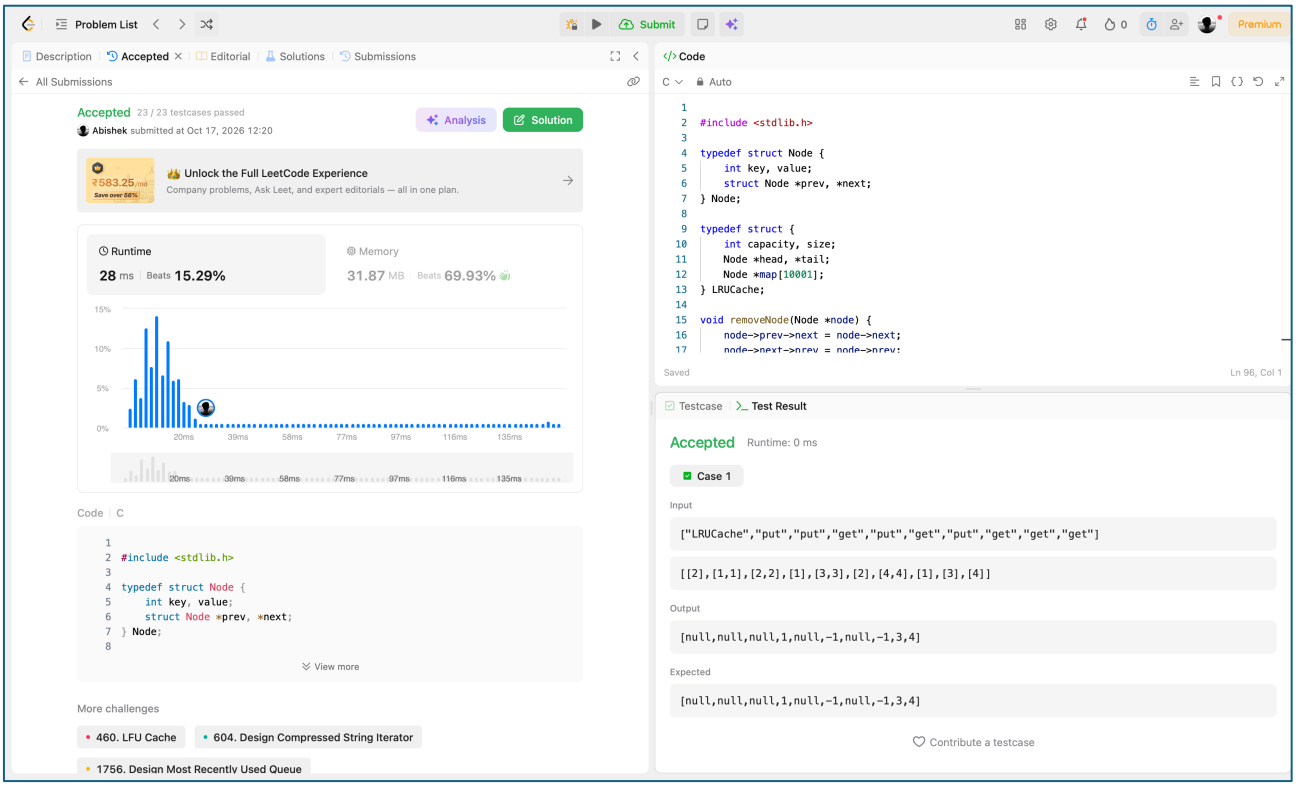
```
    addFront(obj, node);
    obj->map[key] = node;
    obj->size++;
}

void lRUCacheFree(LRUCache* obj) {
    Node *curr = obj->head;

    while (curr != NULL) {
        Node *next = curr->next;
        free(curr);
        curr = next;
    }

    free(obj);
}
```

Output:



Ex 7 :

7) LRU Cache

Design a data structure that follows the constraints of a **Least Recently Used (LRU) cache**.

Implement the LRUCache class:

- LRUCache(int capacity) Initialize the LRU cache with **positive** size capacity.
- int get(int key) Return the value of the key if the key exists, otherwise return -1.

- void put(int key, int value) Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, **evict** the least recently used key.

The functions get and put must each run in O(1) average time complexity.

Ex 8 :

8) LFU Cache

Design and implement a data structure for a Least Frequently Used (LFU) cache.

Implement the LFUCache class:

- LFUCache(int capacity) Initializes the object with the capacity of the data structure.
- int get(int key) Gets the value of the key if the key exists in the cache. Otherwise, returns -1.
- void put(int key, int value) Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the **least frequently used** key before inserting a new item. For this problem, when there is a **tie** (i.e., two or more keys with the same frequency), the **least recently used** key would be invalidated.

To determine the least frequently used key, a **use counter** is maintained for each key in the cache. The key with the smallest **use counter** is the least frequently used key.

When a key is first inserted into the cache, its **use counter** is set to 1 (due to the put operation). The **use counter** for a key in the cache is incremented either a get or put operation is called on it.

The functions get and put must each run in O(1) average time complexity.

Program:

```
#include <stdlib.h>

#define MAX_KEY 100001
#define MAX_FREQ 200005

typedef struct Node {
    int key, value, freq;
    struct Node *prev, *next;
} Node;

typedef struct {
    Node *head, *tail;
} List;

typedef struct {
    int capacity, size, minFreq;
    Node *map[MAX_KEY];
    List *freqList[MAX_FREQ];
} LFUCache;

Node* createNode(int key, int value) {
    Node *node = malloc(sizeof(Node));
```

```
    node->key = key;
    node->value = value;
    node->freq = 1;
    node->prev = node->next = NULL;

    return node;
}

List* createList() {
    List *list = malloc(sizeof(List));

    list->head = calloc(1, sizeof(Node));
    list->tail = calloc(1, sizeof(Node));

    list->head->next = list->tail;
    list->tail->prev = list->head;

    return list;
}

void removeNode(Node *node) {
    node->prev->next = node->next;
    node->next->prev = node->prev;
}

void addFront(List *list, Node *node) {
    node->next = list->head->next;
    node->prev = list->head;

    list->head->next->prev = node;
    list->head->next = node;
}

void updateFreq(LFUCache *obj, Node *node) {
    int freq = node->freq;
    List *oldList = obj->freqList[freq];

    removeNode(node);

    if (freq == obj->minFreq &&
        oldList->head->next == oldList->tail) {
        obj->minFreq++;
    }

    node->freq++;

    if (obj->freqList[node->freq] == NULL) {
        obj->freqList[node->freq] = createList();
    }

    addFront(obj->freqList[node->freq], node);
}

LFUCache* lFUCacheCreate(int capacity) {
    LFUCache *obj = calloc(1, sizeof(LFUCache));

    obj->capacity = capacity;
    obj->size = 0;
    obj->minFreq = 0;
```

```
    return obj;
}

int lFUCacheGet(LFUCache* obj, int key) {
    Node *node = obj->map[key];

    if (node == NULL)
        return -1;

    updateFreq(obj, node);

    return node->value;
}

void lFUCachePut(LFUCache* obj, int key, int value) {
    if (obj->capacity == 0)
        return;

    Node *node = obj->map[key];

    if (node != NULL) {
        node->value = value;
        updateFreq(obj, node);
        return;
    }

    if (obj->size == obj->capacity) {
        List *list = obj->freqList[obj->minFreq];

        Node *lru = list->tail->prev;

        removeNode(lru);
        obj->map[lru->key] = NULL;

        free(lru);
        obj->size--;
    }

    node = createNode(key, value);

    if (obj->freqList[1] == NULL) {
        obj->freqList[1] = createList();
    }

    addFront(obj->freqList[1], node);

    obj->map[key] = node;
    obj->minFreq = 1;
    obj->size++;
}

void lFUCacheFree(LFUCache* obj) {
    for (int i = 0; i < MAX_FREQ; i++) {
        List *list = obj->freqList[i];

        if (list == NULL)
            continue;
```

```
        Node *curr = list->head;

        while (curr != NULL) {
            Node *next = curr->next;
            free(curr);
            curr = next;
        }

        free(list);
    }

    free(obj);
}
```

Output:

